CrunchBite Corp — Enterprise Technology Context

Shared Platform Overview

All CrunchBite product teams (Digital Ordering, Loyalty & Rewards, Kitchen & Operations) build on a common AWS-hosted microservices platform. Each team owns its own services but depends on shared infrastructure, shared event bus, and shared identity layer.

Shared Infrastructure

Cloud & Deployment

- **Cloud provider**: AWS — us-east-1 (primary); us-west-2 (failover for Order and Payment services)
- **Runtime**: Node.js 20 LTS on AWS Lambda (arm64 Graviton3); all services deployed as container images via Amazon ECR
- **IaC**: AWS CDK v2 TypeScript — all resources version-controlled; no click-ops permitted
- **CI/CD**: GitHub Actions — lint → unit test → integration test → E2E (Playwright) → deploy staging → manual promote to production
- **Environments**: development, staging (always on), production; feature branches deploy to ephemeral environments

API Gateway & Security

- **API Gateway**: AWS API Gateway (HTTP API) + CloudFront CDN for customer-facing endpoints
- **Auth**: JWT RS256 tokens via **AWS Cognito User Pools**; guest checkout uses a scoped JWT (4-hour TTL) for unauthenticated ordering
- **WAF**: AWS WAF with OWASP Top 10 rules applied at CloudFront distribution
- **mTLS**: enforced on B2B integrations (Otter webhooks, Adyen callbacks, MICROS sync)
- **Rate limiting**: applied at API Gateway per `clientId`; burst limits configurable per endpoint

Event Bus

- **Apache Kafka (Amazon MSK 3.6)** is the enterprise event bus; all domain events are published here
- Key events cross-team teams must not call each other's REST APIs directly for data sync — they consume events):

Event	Producer	Key Consumers
`order.placed`	Order Service	Kitchen Service, Loyalty Service, Analytics
`order.ready`	Kitchen Service	Notification Service, Analytics
`item.86d`	Menu Service	Order Service (order validation), KDS
`points.earned`	Loyalty Service	Notification Service
`offer.used`	Loyalty Service	Order Service (validation)
`payment.completed`	Payment Service	Order Service, Loyalty Service
`menu.updated`	Menu Service	All ordering channel services
`shift.opened`	Store Config Service	Analytics, Kitchen Service

Observability

- **AWS CloudWatch + X-Ray**: service metrics, traces, and structured logs
- **Datadog APM**: cross-service distributed tracing; used for performance profiling
- **PagerDuty**: on-call alerting; P1 (revenue-impacting) pages the on-call engineer within 5 minutes
- **Dashboards**: real-time operational dashboards in CloudWatch; franchise and SOS dashboards in Tableau (Snowflake source)

Data

- **Amazon Aurora PostgreSQL 15 Serverless v2**: transactional data (orders, loyalty, catering)
- **Amazon DynamoDB**: low-latency catalogue data (Menu Service — global tables for multi-region)
- **ElastiCache Redis 7**: menu cache (TTL 60 s), session store, idempotency keys for payments
- **Amazon MSK (Kafka)**: domain event stream; Kafka Connect to Snowflake for analytics pipeline
- **Snowflake**: enterprise data warehouse; source for Tableau franchise dashboards and operational analytics

Feature Flags

- **LaunchDarkly**: used to gate LTO activations, new menu features, kiosk UI experiments, and gradual rollouts per store or region

---

Shared Services (owned by Platform team)

Service	Responsibility
Identity Service	Customer registration, login, MFA, guest checkout tokens, social sign-in (Cognito + Lambda)
Notification Service	Push (FCM/APNs), SMS (Twilio), email (SendGrid), in-app inbox (Lambda + SQS FIFO + Braze SDK)
Analytics Service	Real-time SOS metrics, item-level sales, franchise roll-up, Snowflake feed (Kafka → Snowflake Connector)
Store Config Service	Store hours, delivery radius, price overrides, feature flags per store (Lambda + DynamoDB)

Engineering Standards

- **Languages**: TypeScript (Node.js services and frontends); no new Python or Java services without Architecture Board approval
- **Testing**: unit ≥ 80%, integration ≥ 60%; Playwright E2E on critical ordering and payment paths; all tests run in CI
- **Secrets**: AWS Secrets Manager; Adyen, Braze, MICROS credentials rotated quarterly; no secrets in environment variables or code
- **Idempotency**: all payment and points-earning operations keyed by `orderId`; duplicate Kafka events or retried API calls must not double-charge or double-award
- **Deployments**: no deployments during peak trading hours (11 AM–2 PM and 5 PM–8 PM local store time) without incident commander approval